

COMP90038 Algorithms and Complexity

Decrease-and-Conquer-by-a-Constant

Junhao Gan

Week 5 Lecture 9

Semester 1, 2026

Topics for Today

In this lecture, we will discuss the **Decrease-and-Conquer** approach for algorithm design.

Specifically, we will focus on *Decrease-and-Conquer-by-a-Constant* first.

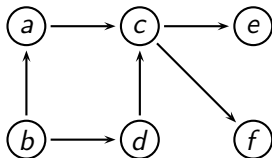
The basic idea is to decrease the problem size iteratively and solve the bigger problem based on the solutions to the smaller ones.

Topological Sorting

A **topological order** of the nodes in a DAG is often considered as a *linearization* of the graph.

Topological orders are found useful in many tasks scheduling problems, where a task can be conducted only when all its “pre-requisites” are done.

For example, subject enrolments.



Topological Sorting

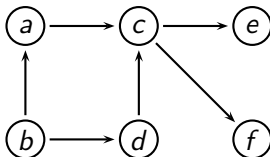
Given a **directed acyclic graph** (DAG) $G = \langle V, E \rangle$, the goal of the **Topological Sorting** problem is to sort all the nodes in V such that:

- for *every* edge $(u, v) \in E$, the node u must *be listed before* v in the order.

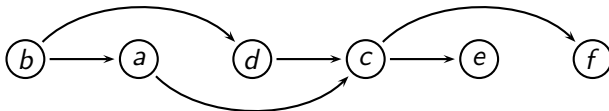
And such a feasible node ordering is called a **topological order** of the nodes.

Topological Sorting: Example

There are four different ways to linearize the following graph.



Here is one:



A directed graph G admits a topological order if and only if G has no cycle.

Topological Sorting by DFS

We can solve the topological sorting problem with depth-first search:

- 1 Pick an arbitrary node u with **no incoming edge**, i.e., $\deg_{\text{in}}(u) = 0$;
- 2 Perform DFS from u and record the *traversed order* in which nodes are popped off the stack;
- 3 List the nodes in the **reverse** of that order.

Question: What is the time complexity of the above algorithm?

$$O(|V| + |E|)$$

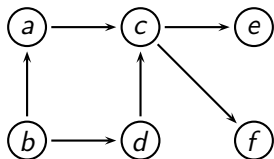
Topological Sorting by DFS

The **correctness** of the above algorithm comes from the following two observations:

- In a DAG, a node v is traversed (i.e., popped out from the stack) **if and only if** all the nodes that are *reachable* from v have been traversed.
- Taking the “reverse popping order” ensures that v is listed before all its reachable nodes.

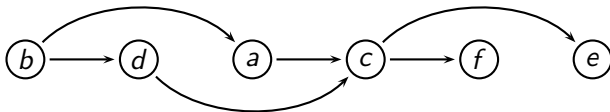
Topological Sorting by DFS: Example

Performing DFS from b and resolving ties by using alphabetical order, the graph gives rise to the traversal stack shown on the right (the popping order shown in red):



$e_{3,1}$ $f_{4,2}$
 $c_{2,3}$ $d_{6,5}$
 $a_{1,4}$ $b_{5,6}$

Taking the nodes in reverse popping order yields b, d, a, c, f, e .



Topological Sorting by a Decrease-and-Conquer Approach

Given a DAG $G = \langle V, E \rangle$, a topological order of V can be constructed *incrementally* by the following way:

- pick an arbitrary node u with *no incoming edge* in G ;
- **append** u to the current topological order;
- **remove** u and *all* its incident edges from G ;
- compute a topological order on (**the new**) G ;

Observe that if a node u with *no incoming* edge, then it is *safe* to **append** u in the current topological order. **Why?**

The above algorithm is a decrease-and-conquer approach.

Specifically, in each iteration, the number of nodes in G is decreased by 1.

Topological Sorting by a Decrease-and-Conquer Approach

A non-recursive version of the above algorithm works as follows.

Let $G' \leftarrow G$. Repeat the following steps until G' is *empty*:

- pick an arbitrary node u with *no incoming edge* in G' ;
- append u to the current topological order;
- remove u and all its incident edges from G' ;

Question: What running time complexity of this algorithm can be?

At first glance, in each iteration, finding a node u with no incoming edge takes $O(|V(G')|)$ time, and removing u from G' takes $O(\deg_{\text{out}}(u))$ time, where $V(G')$ denotes the set of nodes in G' .

Since there are $|V|$ iterations, the overall running time is bounded by $O(|V|^2 + \sum_{u \in V} \deg_{\text{out}}(u)) = O(|V|^2 + |E|) = O(|V|^2)$.

Topological Sorting: An Improved Algorithm

There exists a *simple data structure* which:

- consumes $O(|V|)$ space;
- supports finding a node u with in-degree 0 in $O(1)$ time;
- can be maintained in $O(\deg_{\text{out}}(u))$ time in each iteration.

Therefore, the overall running time of the above algorithm can be improved to $O(|V| + \sum_{u \in V} \deg_{\text{out}}(u)) = O(|V| + |E|)$.

Exercise: Design such a data structure and the algorithm.

Hint: Maintaining the in-degrees of all the nodes in an array might be helpful.